D4 Testing

Primary: Samuel Knight, Anexa Bijoy
Secondary: Wasif Mustafa
Tertiary: Leo Durnion, Hamza Ahmad, Emma Fuller

D4a - Testing Methods and Approaches

We decided to use three different testing methods to ensure the project was functional and met all the requirements.

The majority of tests carried out were unit tests as these allowed us to cover all possibilities by testing isolated components of the game's code. This made it easier to identify and fix errors early on while ensuring each function behaved as expected. We prioritised test cases where the core gameplay features had to be tested e.g. FR_ASSESSMENT1_EVENTS, allowing us to reduce the risk of major faults occurring later on while testing. We implemented these unit tests using the Junit framework alongside the libGDX headless backend which allowed tests to be executed without depending on the graphical elements to be rendered. It also ensured that all the individual unit tests were able to run correctly. This method was appropriate for our project because it allowed us to efficiently test parts of our project without needing to run the full system.

After carrying out the unit tests, we moved onto integration testing to ensure that the individual components worked correctly when combined. This testing method was appropriate for our project because it focused on the interaction between different components of the system, for example whether a player's input is processed by the game logic and passed onto the screen correctly. Integration testing allowed us to identify issues that were not caught during unit testing and ensured the overall game functioned as required after combining all the game elements together.

Finally, end-to-end testing was carried out to ensure that the game functioned correctly as a complete system. We approached this by running the full game and playing through each core feature making sure the game flowed as expected. This testing was conducted as black-box testing, so we focused more on user interactions rather than testing based on the code structure. All group members and external users played through the game to identify any bugs or areas for improvement. Using this feedback we made necessary changes and improvements to produce a final product that met all user and functional requirements.

Testing was carried out continuously throughout development with unit tests being run automatically on every pull request to allow us to identify any problems before they were merged into the master branch. Manual testing was used alongside this to evaluate gameplay and user experience, particularly during the user evolution and feedback sessions. Combining both manual and automated tests with our chosen testing methods was appropriate as it ensured the project remained practical and efficient for the scale of our project.

D4b - Test Report

Unit Tests

By the end of the assessment, we had a total of **119 unit tests**, all of which pass.

By the end of the project, we achieved a **code coverage of 76%**. We were not aiming for 100% coverage since that is usually unrealistic and somewhat unnecessary, so a ratio this close to 80% is something we are satisfied with.

Whilst this number is high, there are certainly areas that would have been nice to test more thoroughly. For example, some interaction testing on the UI elements like in the main menu or `UsernameScreen` would have ensured that nothing broke the user's core interaction with the game.

Despite this, **verification** went very well: We wrote tests that cover 27 out of the total of 31 requirements, giving us a **test coverage of 87%**. The four untested requirements were as follows:

Requirement	Reason for being untested
FR_MUSIC UR_MUSIC	The way music, and sound in general, is handled in the game is somewhat of a mess left by the team in Assessment 1. Because of that, attempting to infer the intended functionality, then testing that that functionality behaves as expected, would be borderline impossible due to the lack of a solid architecture through which we could test. Ideally, we'd like to rewrite the way sound works entirely with a dedicated `SoundHandler` class (or similar), following that, we could implement tests much easier.
FR_MUSIC_MUTE	For similar reasons to above, we never got round to implementing the functionality to mute the music. Due to sound handling being entirely decentralised (it's handled internally, and in a unique way in every class) it would be very difficult to implement this functionality without rewriting the sound system. Since the functionality was never implemented, we never wrote unit tests either.
UR_FILE_SIZE	We were unable to determine a suitable functional requirement to match this User Requirement. Originally, there was a requirement listed to keep the file size under a certain number of megabytes. We suspected that this may not have been what the client had originally intended, so we asked for clarification and were told not to worry about it, so long as we can submit the assessment. Obviously, we've submitted the assessment, so I suppose the requirement is met even though there is no formally defined test.

Manual Tests

Some requirements were unrealistic to be tested automatically, and as such, we wrote precise protocols for manual tests which are laid out on our website: whales-eng1.github.io/WHALES-Website-Assessment-2/testing.html#manual

We also assigned responsible individuals to each manual test to ensure that it was clear who would need to carry out those tests. Where possible, these were carried out after they were written to give us a guide to see what we needed to fix, as well as towards the end of the project to confirm that we had made progress and made each test pass.

Some of the manual tests relied on User Evaluation, and as such could only be carried out once (during the user evaluation stage of the project). The results of this led to some tests failing and changes being made to amend those. The solutions for each problem are available to view in the User Evaluation deliverable.

Although not strictly defined, we obviously have completed end-to-end tests as well by playing through the game from start to finish, as well as interacting with each of the events and achievements to at least informally determine whether they behave as expected.

By the end of the project, all of the manual tests were passing with no notes.

Conclusion

We are not aware of any issues regarding the correctness of our tests, but of course we cannot prove absolute correctness through testing. Since we did not (and cannot) test every possible scenario, it's possible that there are some that lead to a system behaving unexpectedly despite every test passing. We can say with a relatively high degree of certainty that the program adheres to the specifications described in our requirements.

The full results and reports for our testing are available on the website here: https://whales-eng1.github.io/WHALES-Website-Assessment-2/testing.html

Or for direct links to each report:

Unit Test Summary	https://whales-eng1.github.io/WHALES-Website-Assessment-2/reports/tests/index.html
JaCoCo Code Coverage	https://whales-eng1.github.io/WHALES-Website-Assessment-2/reports/jacoco/index.html
CheckStyle (Not strictly testing, but this was the best place to leave this report)	https://whales-eng1.github.io/WHALES-Website-Assessment-2/reports/checkstyle/main.html